

Architecture Selection & Justification

Database Query Performance Profiler — Problem Statement #44

We chose Microservices Architecture for the Database Query Performance Profiler system.

The system ingests high-volume slow-query logs continuously, performs comparatively heavy analysis on execution plans, and produces both a periodic weekly digest and a live dashboard. These three workloads have very different throughput, latency, and failure characteristics, which is the central reason a single, uniformly-scaled architecture is a poor fit here.

Why This Architecture Fits the Scenario

Reason 1 — Independent scaling for a high-throughput component. Log ingestion has to sustain up to 5,000 slow-query records per minute without dropping entries or slowing the monitored databases (NFR-001). That workload is fundamentally different from the rest of the system, which is either periodic (the weekly digest) or interactive and comparatively low-volume (the dashboard). As a standalone Log Ingestion Service, it can be given dedicated resources and scaled on its own, instead of a shared monolith having to be sized for its busiest component at all times.

Reason 2 — Fault isolation around the riskiest logic. Parsing SQL execution plans and generating index recommendations (FR-001) is the most complex, failure-prone part of the system — malformed plans, unsupported SQL dialects, and edge cases are expected in production. Keeping the Execution Plan Analyzer Service separate means a crash or bug there doesn't take down log ingestion or the live dashboard that the DBA and Backend Lead rely on day to day. Each service can also be built, tested, and redeployed on its own schedule without a full-system release.

Component Responsibilities

Component	Responsibility	Addresses
Log Ingestion Service	Streams and normalizes slow-query logs from Postgres and MySQL sources in real time.	FR-002, NFR-001
Query Log Store	Durable storage for raw logs and parsed execution plans; single source of truth for the rest of the system.	FR-002, NFR-002
Configuration Service	Stores and serves the per-instance slow-query threshold used to classify incoming queries.	FR-005
Execution Plan Analyzer Service	Parses EXPLAIN plans, detects sequential scans, and recommends missing composite indexes.	FR-001
Digest Generator Service	Assembles the weekly optimization digest and hands it off for delivery.	FR-003
Dashboard Service	Serves the on-demand view of top slow queries for the DBA and Backend Lead.	FR-004

Architecture Comparison

Criterion	Layered	Client-Server	Microservices (chosen)
Scaling ingestion independently	Poor — whole app scales as one unit.	Poor — server is a single scaling unit.	Strong — ingestion scales on its own.
Fault isolation	Weak — a crash in one layer risks the process.	Weak — server is a single point of failure.	Strong — one service failing doesn't stop the rest.
Operational simplicity	Strong — one deployable unit.	Strong — centralized, simple to run.	Weaker — more services to deploy and monitor.
Fit for this scenario	Ingestion throughput would be bottlenecked by the whole app.	Single server can't isolate heavy batch digest work from live traffic.	Matches the very different load profiles of ingestion, analysis, and reporting.

Security Advantage

Each microservice connects to the shared Query Log Store with its own narrowly scoped credentials — the Log Ingestion Service gets write-only access, while the Analyzer and Dashboard services get read-only access. Query logs can expose table structure, filter values, and business logic, so this least-privilege separation means a compromise of any single service does not grant full read/write access to sensitive query data across the whole system.

Performance Benefit

Because the Digest Generator Service and Dashboard Service are decoupled from the Log Ingestion Service, the weekly digest job — which scans a full week of accumulated data and is comparatively heavy — can run without competing

for resources with real-time log ingestion. This keeps ingestion latency low and the dashboard responsive even while a digest is being generated.

Trade-offs We Accepted

Microservices add real operational cost — more services to deploy, monitor, and keep consistent, plus network latency between them that a monolith wouldn't have. We judged this acceptable here because the system has a small, fixed set of services (six), each with a clearly distinct responsibility and load profile, so the added complexity buys real isolation rather than being complexity for its own sake.